

To exit full screen, press and hold Esc



Application Development Fundamentals

Module 7: Importing and integrating

In **Module 7**, we will cover two different methods to bring external data into ServiceNow.

Module objectives

- ✓ Import records from an Excel file.
- ✓ Use web services to integrate to an external data source.

Labs and activities

- Lab 7.1.1 Importing records from a spreadsheet
- Lab 7.2.1 Web service consumer

Import data

Module 7.1

Import Sets are a common way to pull data into ServiceNow.

Application development checklist

Define Process	Design and Build Data Model	Automate
✓ Business Problem	✓ Tables	✓ Flow Designer
✓ Outcomes	✓ Columns	✓ Scheduled Script Execution
✓ Input(s)	✓ Relationships	✓ Email
✓ Output(s)	Design and Create User Interface	Integrate
✓ User personas/ stakeholders	✓ Desktop/ Tablet	☐ SOAP/ REST/CSV/ Microsoft Excel
✓ Process steps	Apply Security	Enhance User Interface
	✓ Roles	☐ Service Catalog
	✓ Access Control	Test

Although there are more considerations than those listed here, this checklist is a good foundation for preparing to create an application.

Integration interfaces



ServiceNow provides a number of **integration interfaces** to the platform. Many factors should be considered when selecting the appropriate integration method.



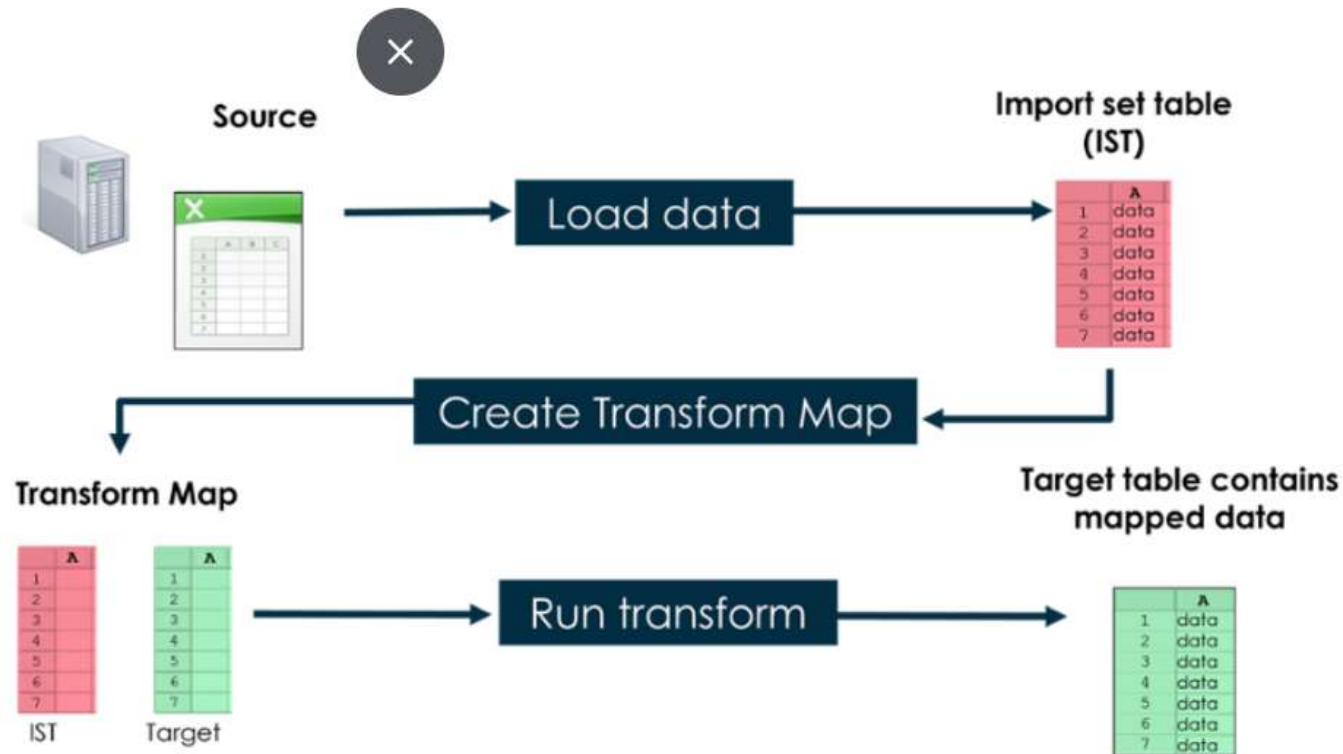
Before importing any data

Time spent planning and examining data **before** import will save time and potential problems later.

- Understand what data is coming in and where it should be placed.
- Plan time before an import to verify the data.
 - Correct inaccurate data.
 - Delete undesired or inaccurate data.

	A	B	C
1	Insured	Type	Short Description
2	Abel Tuter	1	Dog bite
3	Adela Cervantsz	1	Slip and fall
4	Aileen Mottern	2	Single car accident with tree
5	Alejandro Mascall	1	Fall off ladder
6	Alene Rabeck	2	Two car accident on highway 1
7	Aileen Mottern	2	Single car accident with tree
8	Abel Tuter	1	Dog bite

Process for importing via import set



Import sets provide a mechanism to pull data into ServiceNow. Import sets store data in import set tables. The import process skips records when the data in the instance is the same as the data being imported.

Transform Maps determine the relationships between fields in an import set table to fields in an existing ServiceNow table.

The **target table** is the table where the data will be placed, post-transformation.

Loading data into an import set table – Excel

When loading data into an import set table using Excel:

- Data is loaded into a staging table and not directly into the final destination.
- All content from a worksheet is loaded into the staging table.
- Not all data from the staging table has to be copied to the final destination.



Create a new import set table:

1. In the main ServiceNow browser window, open **System Import Sets > Load Data**.
2. Select the **Create table** option.
3. Provide a name for the Import Set Table by putting a value in the **Label** field. The Name field is automatically populated.
4. Select **Choose File**.
 - a. Navigate to the file you want to upload.
 - b. Select **Open**.
5. Set the **Sheet number**.
6. Set the **Header row** number.
7. Select **Submit**.



Create Transform Map

A Transform Map **maps data from the source (import set) table to the target table.**

A Transform Map is a **set of field maps that determine the relationships between fields in an import set and fields in an existing ServiceNow table** (such as Incidents or Users). Once defined, existing Transform Maps can be reused for mapping data from an import set to a ServiceNow table.

The screenshot shows the 'Table Transform Map' configuration interface in ServiceNow. The title bar indicates 'Table Transform Map' and 'New record'. The form is divided into two main sections. The left section contains fields for:

- Name:** 'Import Old Claims Records' (text input)
- Source table:** 'Old claims [x_snc_claims_old_cl...]' (dropdown menu)
- Active:** checked (checkbox)
- Run business rules:** checked (checkbox)
- Enforce mandatory fields:** 'No' (dropdown menu)
- Copy empty fields:** unchecked (checkbox)
- Create new record on empty coalesce fields:** unchecked (checkbox)

 The right section contains:

- Application:** 'Claims' (dropdown menu)
- Created:** empty (text input)
- Target table:** 'Claims [x_snc_claims_claims]' (dropdown menu)
- Order:** '100' (text input)
- Run script:** unchecked (checkbox)

Create a Transform Map:

1. After importing data, select the **Create transform map** Related Link or create a Transform Map in ServiceNow Studio.
2. Configure the Transform Map.
3. **Save.**

Name	Provide a name for the Transform Map.
Source table	Select the Import Set (temporary) table.
Target table	Select the table into which records will be copied.
Run business rules	Select to run business rules, workflows, approval engines, auditing, and field normalization while the transformation inserts or updates data into the target table.
Enforce mandatory fields	Choose whether to enforce mandatory fields on the target table.
Copy empty fields:	Choose this option to clear fields with existing values when an incoming field contains an empty value.
Create new record on empty coalesce fields	Choose this option to create a new record when all coalesce fields are empty.

Auto Map/Mapping Assist

Fields on the import set table can be matched to fields on the target table, either automatically (if field names are identical) or manually.



Automatic Mapping Utility – The simplest mapping method is where all of the field names of the import sets match the names of the fields on the target table. In this case, simply select **Auto map matching fields** in the Related List in the Table Transform Map form and confirm proper matching. If there are any discrepancies in terms of how fields were automatically matched, these can easily be corrected using the mapping assist utility.

To automatically match fields, select the **Auto map matching fields** Related Link. In the image above, there was an automatic match of the State fields.

Mapping Assist Utility – Provides a visually intuitive environment for specifying mapping between Import Set fields and production table fields. With the mapping assist utility, it is possible to map a single source field (field on an Import Set table) to multiple destination fields (fields on a production table).

Manually map fields:

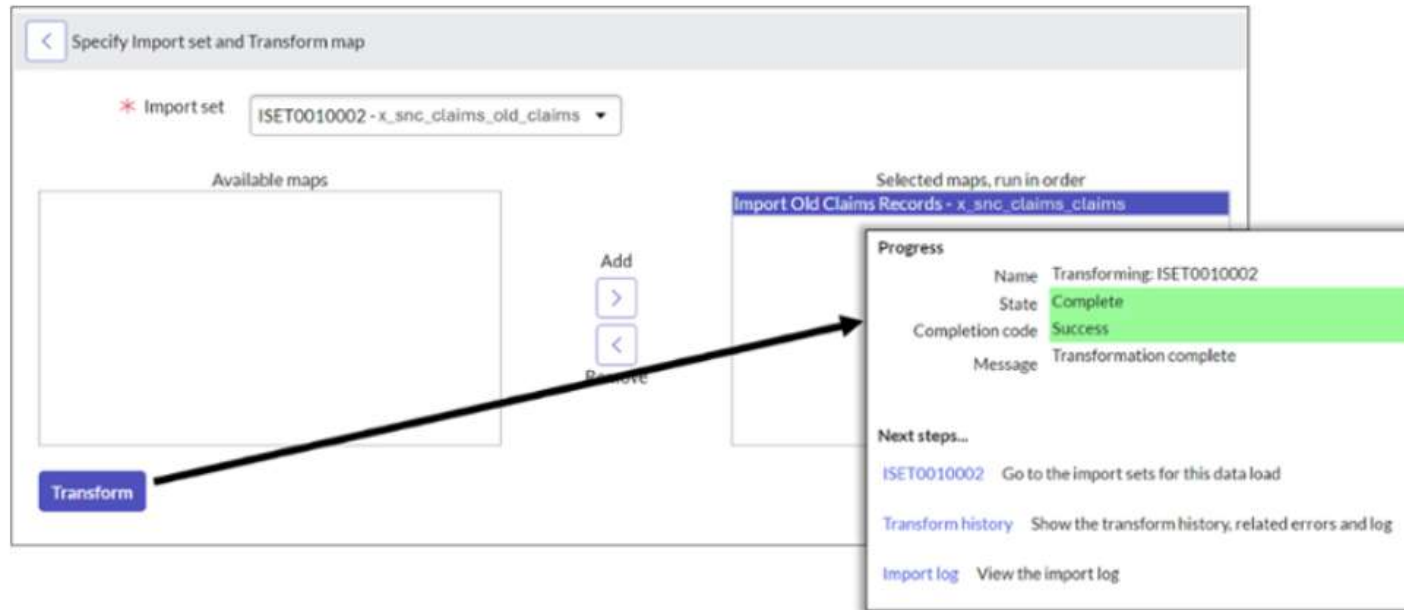
1. Select the **Mapping Assist** link.
2. Select a field from the Source list and select the **Add** button.
3. Select the matching field from the Target list and select the **Add** button.
4. **Save**.

When all fields are matched properly, select **Transform** in the related lists to begin transforming data onto the destination table.



Transform

Transforming copies data from the import set table to the target table using the map(s).



Transform data:

1. Select the **Transform** Related Link.
2. Select the import set to copy records from.
3. Select the map(s) to use when copying.
4. Select the **Transform** button.
5. When the transform is complete, choose a link from the Next steps...section.

ISETXXXXXXX - Go to the import set.

Transform history - Show the transform history, related errors and log.

Import log - View the import log.



Module 7 labs



Time

15–20 min

7.1.1 Importing records from a spreadsheet

- Create a new update set to store changes to the application.
- You will import passwords from a Microsoft Excel file into a new table.

Importing records from a spreadsheet

Lab 7.1.1



Time

15–20 minutes

Lab objectives

You will achieve the following objectives:

- Create a new update set to store changes to the application.
- Import passwords from a Microsoft Excel spreadsheet into a new table.

A. Create an update set

1. In ServiceNow Studio, create and apply a new update set called: **LR-Module 7** to track the changes we make in this module.

(We've done this several times, so try to remember the steps on your own. If you need a reminder, refer to Lab 6.1.1)

B. Preparation

Download the **passwords** Excel file for use during this lab.

A. In the main browser window, open **All > Self-Service > Knowledge**.

Importing records from a spreadsheet

Lab 7.1.1

**Time**

15–20 minutes

Lab objectives

You will achieve the following objectives:

- Create a new update set to store changes to the application.
- Import passwords from a Microsoft Excel spreadsheet into a new table.

A. Create an update set

1. In ServiceNow Studio, create and apply a new update set called: **LR-Module 7** to track the changes we make in this module.

(We've done this several times, so try to remember the steps on your own. If you need a reminder, refer to Lab 6.1.1)

B. Preparation

B. Preparation

Download the **passwords** Excel file for use during this lab.

A. In the main browser window, open **All > Self-Service > Knowledge**.

B. Select the **ADF Knowledge Base**.

C. Select the **Passwords** file, and it will automatically download.

.....

1. Open the **passwords** Excel file and examine its contents.

List the columns from the spreadsheet here:

type response here

Next, you will modify some records to make it easier to locate them later.

2. In the main ServiceNow window, **change the application scope to Global**.

3. Open **Configuration > Base Items > Computers**.

4. Use the **filter** to locate the records for the Asset tags listed in the password Excel file.

Try to figure out the filter settings on your own.

If you need a hint, use the **+ Show hint** button below.

[+ Show hint](#)

5. Modify the name of each computer to make it easier to find the loaner laptops. For example, add **LNR** in front of "MacBook".

Computers

Name

Search

All > Asset tag in P1000503, P1000640, P1000815, P1000814, P1000517, P1000666, P1000871

	Name	Manufacturer	Operating System
	Search	Search	Search
	LNR - MacBook Air 13"	Apple	Mac OS 10 (OS/X)
	LNR - MacBook Air 13"	Apple	Mac OS 10 (OS/X)
	LNR - MacBook Pro 15"	Apple	Mac OS 10 (OS/X)
	LNR - MacBook Pro 15"	Apple	Mac OS 10 (OS/X)
	LNR - MacBook Pro 17"	Apple	Mac OS 10 (OS/X)
	LNR - MacBook Pro 17"	Apple	Mac OS 10 (OS/X)
	LNR - MacBook Pro 17"	Apple	Mac OS 10 (OS/X)

C. Creating a password vault

1. Make sure the scope is still set to **Global**.
2. Open **System Definition > Tables**.
3. Select **New** to create a new table.

Label: **Password Vault**

Name: **u_password_vault** (this field populates automatically)

Create module: **Selected [checked]**

Create mobile module: **Not Selected [unchecked]**

Add module to menu: -- **Create New** --

New menu name: **Password Vault**

Controls tab:

Create access controls: **Selected [checked]**

User role: **admin**

Application Access tab:

Accessible from: **All application scopes**

Can read: **Selected [checked]**

Allow access to this table via web services: **Selected [checked]**

4. **Save** (not Submit).
5. In the Columns tab section, double-click **Insert a new row...** to add two new fields to the **u_password_vault** table.

Column label: **Password**

Type: **String**

Column label: **Asset**

Type: **String**

Columns Controls Application Access						
Table Columns for text Search 1 to 6 of 6 New						
Dictionary Entries						
Column label	Type	Reference	Max length	Default value	Display	
Sys ID	Sys ID (GUID)	(empty)		32	false	
Created	Date/Time	(empty)		40	false	
Updated by	String	(empty)		40	false	
Updates	Integer	(empty)		40	false	
Updated	Date/Time	(empty)		40	false	
Created by	String	(empty)		40	false	
✕ Password	String				false	
✕ Asset	String				false	
Insert a new row...						

6. Select **Update**.

D. Importing passwords from an Excel file

1. In the main ServiceNow browser window, open **System Import Sets > Load Data**.

2. Create a new import set:

Import set table: **Create table**

Label: **Password Import File**

Table Name: **u_password_import_file** (automatically populates)

Source of the import: **File**

File: **passwords.xlsx**

3. Select **Submit**.

4. Select the **Create transform map** link.

Progress

Name	ImportProcessor
State	Complete
Completion code	Success
Message	Processed: 7, inserts 7, updates 0, errors 0, empty and ignored 0, ignored errors 0 (0:00:08.198)

Next steps...

[Import sets](#) Go to the import sets for this data load

[Loaded data](#) Go to the newly imported data inside the staging table: u_password_import_file

[Create transform map](#) Create a transform map for the newly staged data

[Import log](#) View the import log

5. Configure the transform map to map the imported data to the fields in the **u_password_vault** table:

Name: **Password Transform Map**

Source Table: **Password Import File [u_password_import_file]**

Active: **Selected [checked]**

Run business rules: **Unselected [unchecked]**

Target Table: **Password Vault [u_password_vault]**

6. **Save** (not Submit).

7. In the **Related Links** section, select the **Auto Map Matching Fields** link.

Examine the Source and Target fields in the Field Maps Related List section.

Did the fields map correctly? If not, correct any problems.

8. Select the **Transform** link from the **Related Links** section.

9. In the **Specify import set and Transform map form**, select the **Transform** button.

< Specify Import set and Transform map

* Import set ISET0010001 - u_password_import_...

Available maps

Selected maps, run in order

Password Transform Map - u_password_vault

Add

>

<

Remove

Transform

E. Testing

1. In the **Next steps...** section, select the link to go to the import set. The link will have the syntax **ISET001000X**.

Progress

Name	Transforming: ISET0010001
State	Complete
Completion code	Success
Message	Transformation complete

Next steps...

[ISET0010001](#) Go to the import sets for this data load

[Transform history](#) Show the transform history, related errors and log

[Import log](#) View the import log

2. Scroll down to the **Import Set Rows** tab in the **Related Links**. Use the data in this section to verify the import was successful.
3. Open **Password Vault > Password Vaults** to see the list of Assets and Passwords.

Password Vaults		Asset	Search	Actions on selected rows...	New
All					
☐	☐	Asset	Password		
		P1000814	mars		
		P1000517	jupiter		
		P1000640	venus		
		P1000666	saturn		
		P1000815	earth		
		P1000871	uranus		
		P1000503	mercury		

Nice job! In this lab, you accomplished the following:

- You created a new update set to store changes to our application.
- You imported passwords from an Excel spreadsheet into a new table.

Congratulations! You have completed this lab.



Web services

Module 7.2

Web service connections are another way to share data between ServiceNow and third-party services.

User story

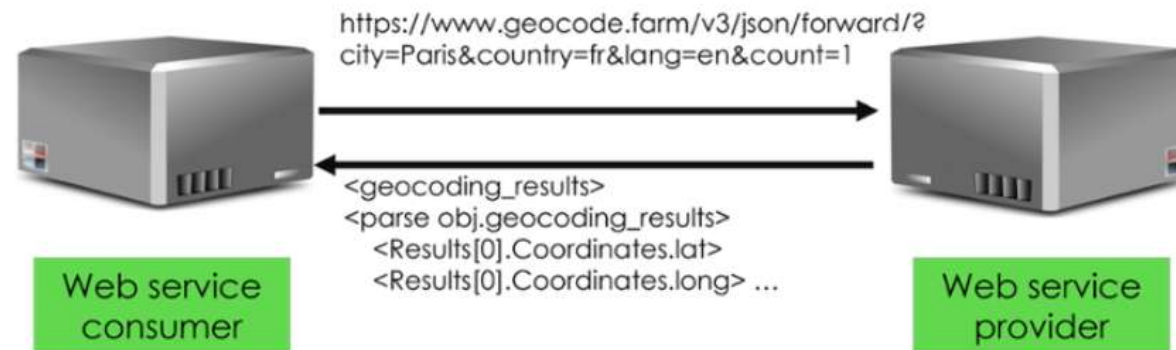
As the **Application Developer**, I need to configure an instance to be a **web service consumer**, so the instance can communicate with and receive data from a **web service provider**.



What are web services?

Web services are:

- A web-based method **allowing applications to connect to other software applications** over a network.
- **An exchange of information** between service providers and service consumers.



HTTP-based web services allow diverse applications to talk to each other. **ServiceNow supports both inbound (provider) and outbound web service (consumer) services.**

Consumers vs. Publishers



Consumer of web services (Consuming)

- Invokes/consumes published web services.
- Consuming a third-party web service from ServiceNow can be accomplished by creating a new *Outbound REST Message*.
- Based on the provided parameters and security restrictions, it can create, read, update, or delete records (CRUD operations).
- ServiceNow can invoke a web service from multiple areas within the platform including, Business Rules, UI Actions, Client Scripts, Workflow, and Flow Designer.

Publisher of web services (Publishing/Producing)

- Publishes web services for clients to invoke (consume).
- Publishing a web service in ServiceNow that can be consumed by a third-party can be accomplished by creating a new *Inbound* web service.
- Publishing a web service means enabling a web service user (consumer) to locate the service description.
- Publishes instructions telling the consumer how they should interact with the web service.

As a web service provider, the Now Platform supports:

- **Direct web services** - Provides access (input and query) to the underlying table structures and their associated data.
- **Web service import set** - An input-only mechanism which invokes data mapping using Transform Maps.
- **Scripted web services** - Develop custom web services executed in JavaScript.

Configuring a REST message

The REST message form can be used to **develop, prototype, and save outbound REST messages**. These messages are reusable in Business Rules, Email Notifications, and other server-side scripts.

The screenshot shows the 'REST Message' configuration form for 'Claim Locations'. The form has a header bar with a menu icon, the title 'REST Message Claim Locations', and an 'Update' button. Below the header, there are several fields: 'Name' (Claim Locations), 'Application' (Claims), 'Accessible from' (This application scope only), 'Description' (Map locations of all open claims), and 'Endpoint' (https://www.geocode.farm/v3/xml/forward). At the bottom, there are two tabs: 'Authentication' (selected) and 'HTTP Request'. Under the 'Authentication' tab, there is a dropdown for 'Authentication type' set to 'No authentication' and a checkbox for 'Use mutual authentication' which is unchecked.

Name	Name of REST Message.
Endpoint	The URL of the data to be retrieved, updated, or deleted.
Authentication type	Choose None, Basic, or OAuth 2.0.
Use mutual authentication	Establishes trust by exchanging Secure Socket Layer (SSL) certificates.
Application	Set the application Scope.
Accessible from	Choose This application scope only or All application scopes.

Configure a web services consumer:

1. In ServiceNow Studio, create a new application file of type **REST Message**.
2. Configure the REST Message.
3. Select **Submit**.

For more detail on different types of authentication, select each button below:

Outbound REST Authentication



REST APIs



Enable OAuth with inbound REST



© 2025 ServiceNow, Inc. All Rights Reserved.

servicenow

HTTP methods





HTTP methods

The HTTP methods **do not appear** on the REST message form **until the record is saved** for the first time. HTTP methods are used to **retrieve, update, create, and delete** data at the endpoint. The user must be authorized to access data.

HTTP Methods			
for text		Search	
REST Message = Claim Locations			
☐	Name	HTTP method	Endpoint
☐	Default GET	GET	https://www.geocode.farm/v3/xml/forward



Examining parameters for an endpoint

As a web service consumer, you must understand what information a web service provider expects. This is provided in the web service's published documentation.

Consumers must provide values for the required variables. In the example shown, `addr`, `key`, `lang`, `country`, and `count` are all parameters that can be passed when the web service is called.

FORWARD GEOCODING REQUESTS:

Forward geocoding takes a provided address or location and returns the coordinate set for the requested location.

- a. Required parameters for Forward Geocoding Requests:
 1. `addr` — The string to search for. Usually a street address. Should be URL encoded prior to sending request.
 2. `key` — API Key. Only Required for Paid Users. If not specified, Free User restrictions will apply. Located in Dashboard for Paid Users. Paid Users should include with every request to avoid Free User limits taking effect. Free Users should not specify or requests may fail due to Invalid API Key response.
- b. Optional parameters for Forward Geocoding Requests:
 1. `lang` — 2 digit language code to return results in. Currently only "en"(English) or "de"(German) supported.
 2. `country` — The country to return results in. Used for biasing purposes and may not fully filter results to this specific country.
 3. `count` — Max amount of results to return. Used for limiting results only and this amount of results will not always be returned. If less results are available then the count specified, all available results will be returned. Default value is 1.

The full URL path can be seen in the following examples:

(JSON) `https://www.geocode.farm/v3/json/forward?addr=530 W Main St Anoka MN 55303 US&country=us&lang=en&count=1`

(XML) `https://www.geocode.farm/v3/xml/forward?addr=530 W Main St Anoka MN 55303 US&country=us&lang=en&count=1`

Giving values to REST message functions

Add variables to the HTTP Query Parameters.

HTTP Query Parameters			
	Name	Value	Order ▲
✕ ✎	addr	\${addr}	100
✕ ✎	count	\${count}	200
✕ ✎	lang	\${lang}	300
✕ ✎	country	\${country}	400

Configure a REST message HTTP query parameter:

1. In the REST message, select the HTTP method of interest.
2. In the **HTTP Request** section, scroll to the **HTTP Query Parameters** section.
3. Provide a **Name** value matching a parameter from the endpoint API documentation. Do **not** include the equals sign (=), which will be inserted automatically at runtime.
4. Add a **Value** for the name. Enclose the Value in \${ }. At runtime, a dynamically determined value is substituted for the Value.
5. **Save.**



Testing REST message functions

When testing your REST message, you will need to configure Variable Substitutions. **There must be a Variable Substitution for all required HTTP Query Parameters.** Once your substitutions have been defined, select the Test link to post the REST message.

The screenshot shows the 'Related Links' section with the 'Test' link highlighted in a green box. Below it, the 'Variable Substitutions (4)' tab is active, displaying a table with the following data:

Name	Escape type	Test value
addr	No escaping	2225 Lawson lane, Santa Clara, CA
count	No escaping	1
country	No escaping	US
lang	No escaping	en



Did it work?

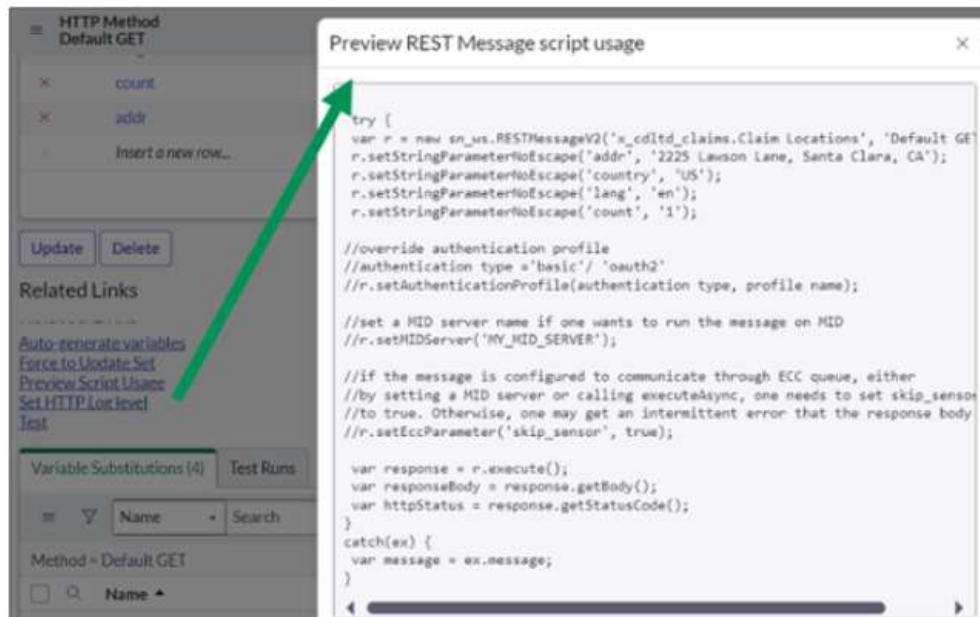
A status of 200 indicates that a connection was made, but you should always check the results to verify you received what was expected.

The Parameters field shows the variables passed at the end of the endpoint (you may have to increase the field size to see all parameters and values).

Response	
Status code	200 OK

Preview script

The server-side code stub is generated automatically, using the hard-coded test variables. **Copy the Preview Message script usage text and paste it into a server-side script.** You will need to **replace the hard-coded test variables** with appropriate dynamic variables. The test does **NOT** extract variable values.



Posting from a script

Tested code stubs can be placed in any server-side script. This allows developers to dynamically pass REST Message Parameter values.

```
1 (function executeRule(current, previous /*null when async*/ ) {
2     var gr = new GlideRecord("x_cdltd_claims_claims");
3     gr.addEncodedQuery('active=true^assigned_to.cityISNOTEMPTY');
4     gr.query();
5     while (gr.next()) {
6
7         var r = new sn_ws.RESTMessageV2('x_cdltd_claims.Claim Locations', 'Default GET');
8         r.setStringParameterNoEscape('addr', '2225 Lawson Lane, Santa Clara, CA');
9         r.setStringParameterNoEscape('country', 'US');
10        r.setStringParameterNoEscape('lang', 'en');
11        r.setStringParameterNoEscape('count', '1');
12
13        var response = r.execute();
14        var responseBody = response.getBody();
15        var httpStatus = response.getStatusCode();
16    }
17 })(current, previous);
```

In this example, the try/catch from the Preview Script Usage has been removed, and the address and country hard-coded values have been modified to be dynamic.

Extracting information from the response

REST Response can be either JSON or XML. **The response is extracted from the returned data.** You must understand the structure of the returned data in order to extract values.

```
//script to parse from JSON response
var obj = JSON.parse(response.getBody());
var myLatjson = obj.geocoding_results.RESULTS[0].COORDINATES.latitude;
var myLongjson = obj.geocoding_results.RESULTS[0].COORDINATES.longitude;

//script to parse from XML response
var xmlDoc = new XMLDocument2();
xmlDoc.parseXML(responseBody);
var myLatxml = xmlDoc.getNodeText("//latitude");
var myLongxml = xmlDoc.getNodeText("//longitude");
```

The JSON Script Include is accessible from all application scopes. It contains methods for working with JSON data.



Integrating to a ServiceNow table

ServiceNow tables can be web service providers. The Application Access tab contains the options to enable/disable web service access. The "Allow access to this table via web services" option is only selectable if the "Accessible from" value is set to "All application scopes".

Table
Claims

* Label

Claims

* Name

x_snc_claims_claims

Extends table

Task

Application

Claims

Columns

Controls

Application Access

Accessible from

All application scopes

Caller Access

-- None --

Can read

☒

Can create

☐

Can update

☐

Can delete

☐

Allow access to this table via web services

☒

Allow configuration

☐

Delete

Update

Delete All Records

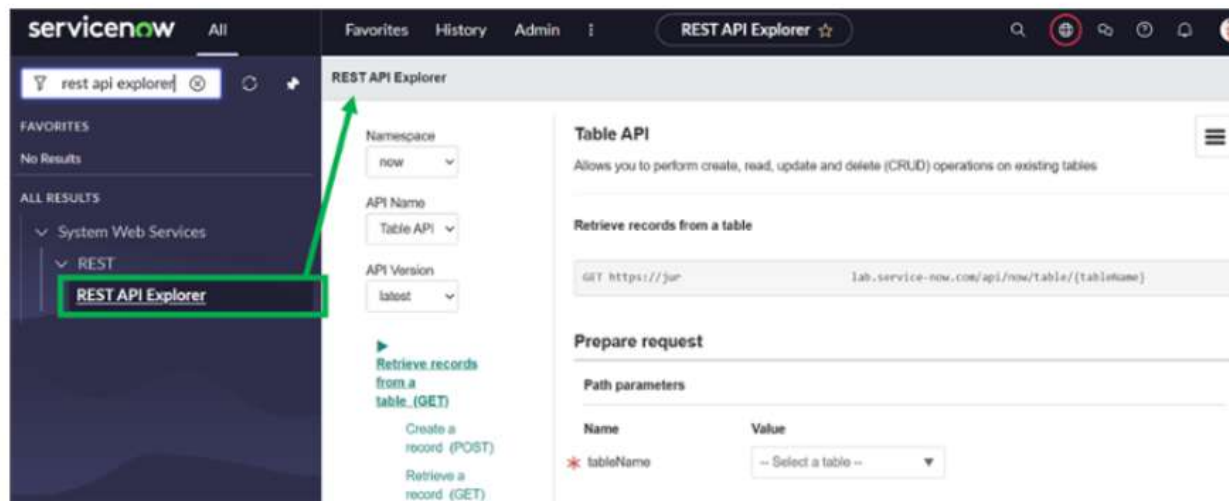


REST API Explorer

The **REST API Explorer** creates **endpoints, methods, and variables** used to send REST requests to a ServiceNow instance.

Important: The REST API Explorer impacts records in the ServiceNow instance database. **DO NOT** practice with the REST API Explorer on a production instance!

In the main ServiceNow window, open **System Web Services > REST > REST API Explorer**.



A REST message is composed of several elements, such as the endpoint and HTTP methods.

HTTP methods interact with records/resources at the REST endpoint:

- **POST** – Create
- **GET** – Read
- **PUT** – Update action to *replace* a record/resource
- **DELETE** – Delete/remove
- **PATCH** – Update action to *modify* a record/resource



Table API

The Table API allows you to query, create, update, and delete a table's records. The Table API is the default API in the REST API Explorer. You cannot change the instance URL when exploring, but you can in the script.

REST API Explorer

Namespace
now

API Name
Table API

API Version
latest

► Retrieve records from a table (GET)

Create a record (POST)

Retrieve a record (GET)

Modify a record (PUT)

Delete a record (DELETE)

Table API

Allows you to perform create, read, update and delete (CRUD) operations on existing tables

Retrieve records from a table

GET https:// /table/{tableName}

Prepare request

Path parameters

Name	Value
* tableName	Password Vault (u_password_vault)

Configure Query Parameters



Configure the endpoint variables using Query Parameters. **The Query Parameters depend on the database operation being performed.** Select the database operation (get, post, put, patch, delete) before configuring.

See the ServiceNow documentation site for information about the Query Parameter fields.

Query parameters

Name	Value	Description
sysparm_query	u_asset=P1000517	An encoded query string used to filter the results
sysparm_display_value		Return field display values (true), actual values (false), or both (all) (default: false)
sysparm_exclude_reference_link		True to exclude Table API links for reference fields (default: false)
sysparm_suppress_pagination_header		True to suppress pagination header (default: false)
sysparm_fields	u_password	A comma-separated list of fields to return in the response
sysparm_limit	1 (Limited to 1 result for testing)	The maximum number of results returned per page (default: 10,000)
sysparm_view		Render the response according to the specified UI view (overridden by sysparm_fields)
sysparm_query_category		Name of the query category (read replica category) to use for queries
sysparm_query_no_domain		True to access data across domains if authorized (default: false)
sysparm_no_count		Do not execute a select count(*) on table (default: false)

Add query parameter

For complex queries, use the **Filter Condition Builder** on a table's list to build the query. After running the query, right-click on the breadcrumb and select **Copy Query**. Pass the query into the **sysparm_query** field.



Testing the web service

To test the web service, select the **Send** button below the Query Parameters section of the REST API Explorer form. Results appear at the bottom of the form. The Response is returned in the format specified in the configuration parameters.

The screenshot shows the 'REST API Explorer' interface. It consists of a top section with a 'Send' button and a bottom section labeled 'Response Body'. An arrow points from the 'Send' button to the 'Response Body' section, indicating the flow of the test execution. The 'Response Body' section displays a JSON object representing the API response.

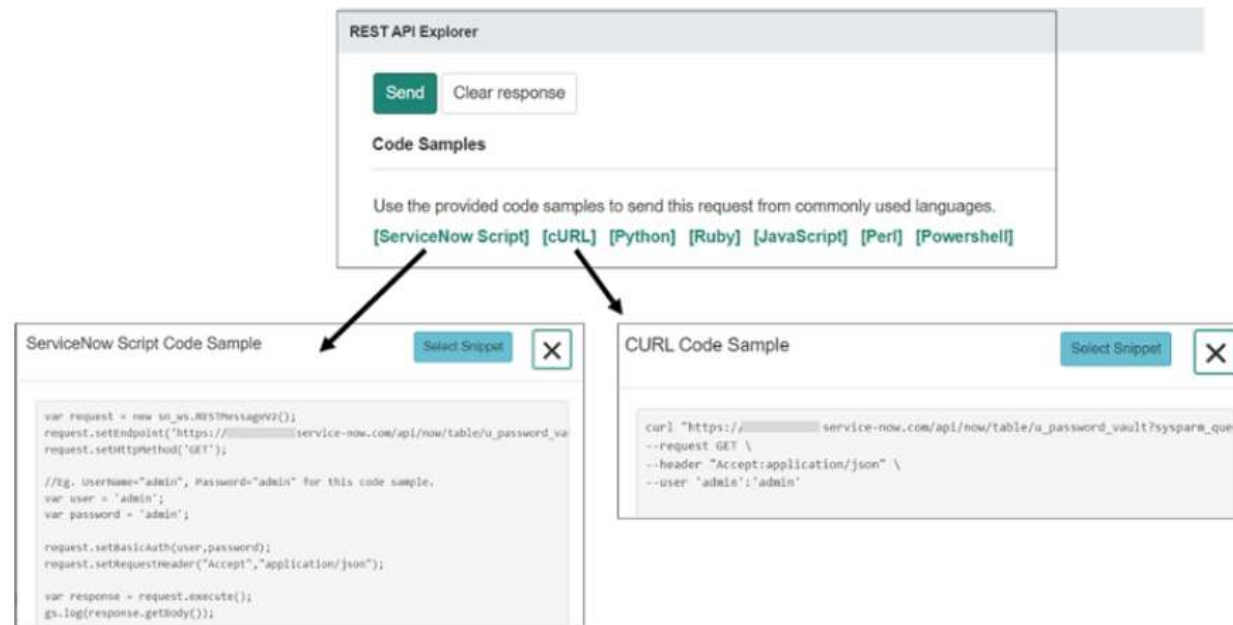
```

{
  "result": [
    {
      "u_password": "jupiter"
    }
  ]
}

```

Scripted API call

The REST API Explorer creates stub code for using REST to interact with ServiceNow tables.



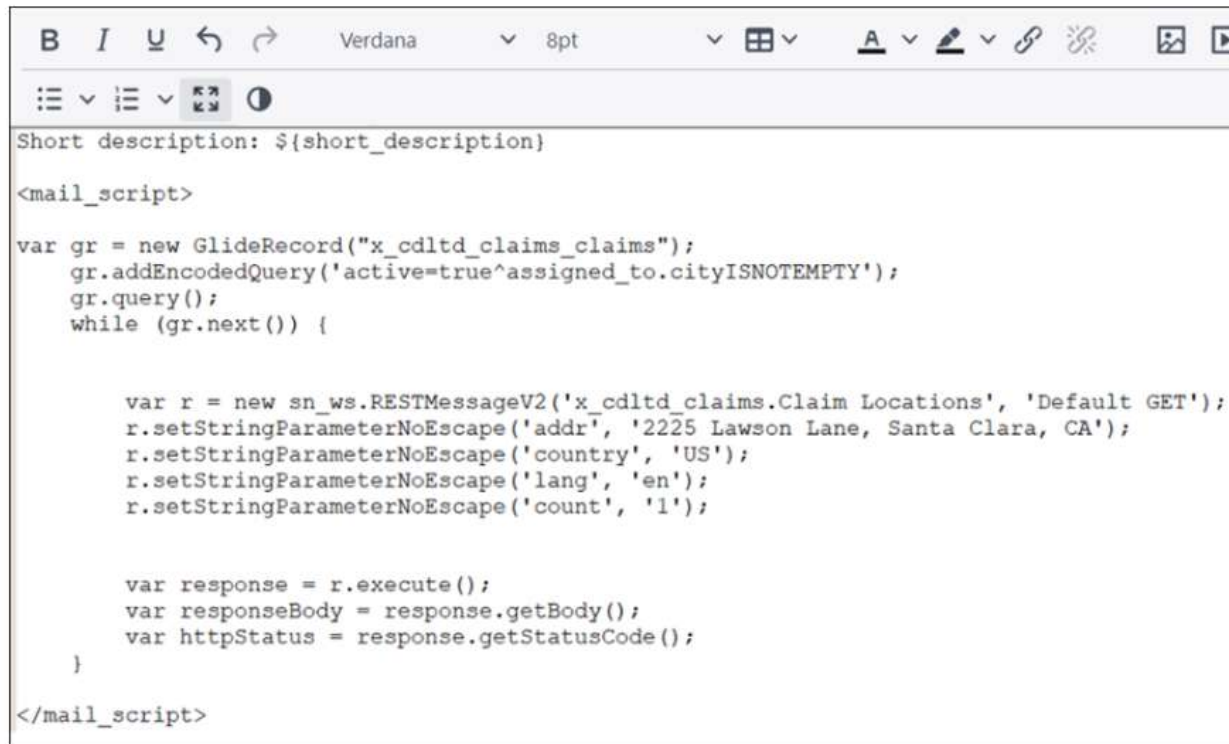
The stub code created by the REST API Explorer does **not** extract information from the web service response. Application developers will have to write the code for processing the response.

Use the **Select Snippet** button to copy the script. Do **not** use your mouse to highlight the script and copy, as that can cause character substitution for some special characters.

Posting from an email notification

When posting from an email notification:

- Enclose the script in `<mail_script>` `</mail_script>` tags.
- Replace any logging or printing methods with the `template.print()` method.

A screenshot of a rich text editor interface. The top toolbar includes bold (B), italic (I), underline (U), undo (↶), redo (↷), font face (Verdana), font size (8pt), text color (A), background color, link, unlink, insert image, and a play button. Below the toolbar is a list of icons for bulleted, numbered, and indented lists, and a help icon. The main text area contains the following code:

```
Short description: ${short_description}

<mail_script>

var gr = new GlideRecord("x_cdltd_claims_claims");
gr.addEncodedQuery('active=true^assigned_to.cityISNOTEMPTY');
gr.query();
while (gr.next()) {

    var r = new sn_ws.RESTMessageV2('x_cdltd_claims.Claim Locations', 'Default GET');
    r.setStringParameterNoEscape('addr', '2225 Lawson Lane, Santa Clara, CA');
    r.setStringParameterNoEscape('country', 'US');
    r.setStringParameterNoEscape('lang', 'en');
    r.setStringParameterNoEscape('count', '1');

    var response = r.execute();
    var responseBody = response.getBody();
    var httpStatus = response.getStatusCode();

}

</mail_script>
```

Note: There is no ability to syntax check a script in the Email Notification Message field. Develop your script in a Business Rule and copy it to the Email Notification after testing.

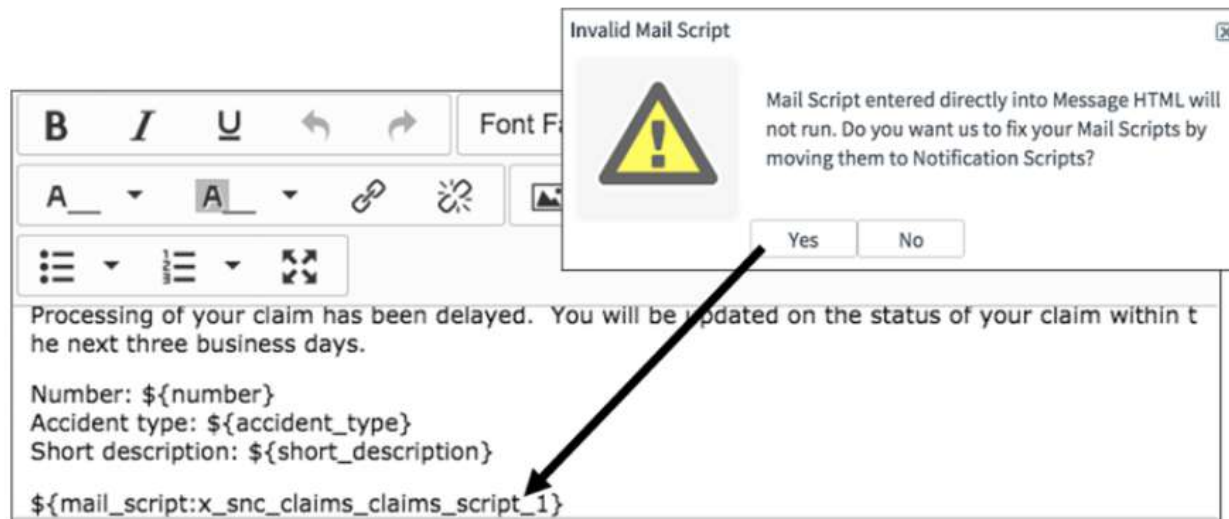
Remember to change any `gs.info()`, `gs.debug()`, `gs.warning()`, or `gs.error()` method calls to `template.print()` in the Email Notification.

Notification script

Notification scripts:

- Are **reusable**.
- Are **not stored** in Notifications.
- Have required syntax **automatically inserted**.

To view or modify Notification scripts, in the main ServiceNow window, open **System Notification > Email > Notification Email Scripts**.



Note: There is **no** mechanism for testing Notification Email Scripts other than generating email. To speed up development and debugging, it is recommended to develop Notification Email Scripts first as Business Rules to allow for easy testing.



Module 7 labs



Time

30–35 min

7.2.1 Web service consumer

- Configure your development instance to be a web service consumer.
- Retrieve passwords from the web service provider.
- Complete the update set.

Note: The technique demonstrated in this lab would normally be used to interact with a *different* instance, but for this activity we will use different tables in the *same* instance.



Web service consumer

Lab 7.2.1



Time

30–35 minutes

Lab objectives

You will achieve the following objectives:

- Configure your Loaner Request application to be a web service consumer.
- Retrieve passwords from the web service provider based on asset tag value and send the passwords to Loaner Item requesters via email.
- Complete the update set.

A. Preparation

Examine the setup for this lab:





You will send a REST request from the consumer to the provider. In this lab, your instance will play the role of BOTH the consumer and the provider.

You will pass the **Asset tag value** from a Loaner Request's Configuration item to the web service provider. The web service provider **will return the password** for the Configuration item if the item is a laptop.

Normally, this technique would be used to interact with *different instances*, but for this activity we will use different tables in the *same* instance.

B. Creating a web service producer

1. In the main ServiceNow window, **confirm you are still in Global scope**.
2. Open **System Web Services > REST > REST API Explorer**. If prompted to explore, select the **Explore** button.
3. Select the **Retrieve records from a table (GET)** link.

REST API Explorer

Namespace:

API Name:

API Version:

Retrieve records from a table (GET) (highlighted)

Create a record (POST)

Retrieve a record (GET)

Modify a record (PUT)

Table API

Allows you to perform create, read, update and delete (CRUD) operations on existing tables

Retrieve records from a table

GET `https://[instance].lab.service-now.com/api/now/table/{tableName}`

Prepare request

Path parameters

Name	Value
* tableName	-- Select a table --

Modify a

4. Configure the Path parameters:

tableName: **Password Vault (u_password_vault)**

5. Configure the Query parameters:

sysparm_query: **u_asset=<the asset value of your choice from the Password Vault table>**
(Make sure there are not any spaces.)

sysparm_fields: **u_password**

6. Configure the Request headers:

Response format: **application/json**

REST API Explorer

Namespace

now

API Name

Table API

API Version

latest

► Retrieve records from a table (GET)

Create a record (POST)

Retrieve a record (GET)

Modify a record (PUT)

Delete a record (DELETE)

Update a record (PATCH)

Export OpenAPI Specification (YAML)

Export OpenAPI Specification (JSON)

Table API

allows you to perform create, read, update and delete (CRUD) operations on existing tables

Retrieve records from a table

GET https://xp3-jan17-516-005.lah.service-now.com/api/now/table/{tableName}

Prepare request

Path parameters

Name	Value
* tableName	Password Vault (u_password_v...

Query parameters

Name	Value	Description
sysparm_query	u_asset=P1000503	An encoded query string used to filter the results
sysparm_display_value		Return field display values (true), actual values (false), or both (all) (default: false)
sysparm_exclude_reference_link		True to exclude Table API links for reference fields (default: false)
sysparm_suppress_pagination_header		True to suppress pagination header (default: false)
sysparm_fields	u_password	A comma-separated list of fields to return in the response
sysparm_limit	1 (Limited to 1 result for testing)	The maximum number of results returned per page (default: 10,000)
sysparm_view		Render the response according to the specified UI view (overridden by sysparm_fields)
sysparm_query_category		Name of the query category (read replica category) to use for queries
sysparm_query_no_domain		True to access data across domains if authorized (default: false)
sysparm_no_count		Do not execute a select count(*) on table (default: false)

Add query parameter

Request headers

Name	Value	Description
Request format	application/json	Format of REST request body
Response format	application/json	Format of REST response body
Authorization	Send as me	Send the request as the current user. To send the request with another user's credentials use the provided code samples, such as cURL.

Add header

Send

C. Testing the web service producer

1. Select the **Send** button at the bottom of the REST API Explorer form.

Scroll down and examine the Response.

Was the test successful?

Was the HTTP Status 200?

Was the correct password returned?

If not, debug and re-test.

D. Using a script in an email notification

1. Select the **[ServiceNow Script]** link in the REST API Explorer.



2. Select the **Select Snippet** button and use **<Ctrl> + C** to copy the code stub to the clipboard, then close the dialog window.
3. In ServiceNow Studio, create a **Business Rule**. (Hint: Create > File > Server Development > Business Rule > Continue.)

Configure the new Business Rule:

Name: **Retrieved Credentials**

Table: **Loaner Request**

Active: **Selected [checked]**

Advanced: **Selected [checked]**

Reflection and notes: Should the Business Rule be before, after, or async?

[+ Show answer](#)

4. On the **When to run** tab, configure when the business rule should run:

When: **async**

Insert: **Selected (checked)**

Update: **Selected (checked)**

5. Select the **Advanced tab**.

Examine the pseudo-code:

If the item type for the current record is cmdb_ci_computer:

Create a new instance of the RESTMessageV2 class

Create an endpoint property on the RESTMessage V2 object

Set the HTTP Request method to Get

Set the credentials for the web service authentication

Set the authorization type

Set the request header

Send the request to the web service

Log the response that is returned by the web service

Set the Parameter u_asset to be the current record's configuration item's asset_tag

Post the function

Store the response body in a variable

Store the response status code in a variable

Create a new instance of the JSON class and decode the response body

Parse the value of the u_password value and store it in a variable

Print "The admin password for your laptop is:" plus the password

6. On the **Advanced** tab, **paste** the script copied to your clipboard into the **Script** field under **// Add your code here.**

Make the following modifications to the stub code:

- Add a condition to the script to test if the value of the *Item type* field is *Laptop*.
Do not use the *Condition* field or the *Filter Condition* field.
- The script will be copied to an *Email Notification*.
Only the contents of the *script* field can be copied.

7. **Modify the code** to meet the condition by adding:

⊗ Hide script

Copy/Paste the following:

```
if (current.item_type == "cmdb_ci_computer") {
```

Hint: Don't forget to put the ending '}' at the end of the code to remove all the errors after adding the new "IF" statement.)

```
(function executeRule(current, previous /*null when async*/ ) {  
  if (current.item_type == "cmdb_ci_computer") {  
  
    var request = new sn_ws.RESTMessageV2();  
    request.setEndpoint('https://lab.service-now.com/api/now/table/
```

8. This JavaScript statement that sends the endpoint, contains a hardcoded asset tag value in the URL. **Modify the statement** to *dynamically* set the asset tag value in the endpoint. Do this by replacing the hardcoded asset tag value with a dynamic variable:

⊕ Show script

Hint: Use the current object and dot-walking.

```
(function executeRule(current, previous /*null when async*/ ) {
  if (current.item_type == "cmdb_ci_computer") {

    var request = new sn_ws.RESTMessageV2();
    request.setEndpoint('https://lab.service-now.com/api/now/table/u_password_vault?sysparm_query=u_asset%3D' + current.cmdb_ci.asset_tag + '&sysparm_fields=u_password&sysparm_limit=1');
    request.setHttpMethod('GET');
```

9. The stub code has script logic to authenticate as admin/admin. **Change the value of the authentication password variable** to the password you use to log into your instance.

```
//Eg. UserName="admin", Password="admin" for this code sample.
var user = 'admin';
var password = 'CHANGE THIS TO YOUR INSTANCE PASSWORD';
```

10. Change any occurrence of **gs.log** to **gs.info**.

Note: gs.log is not part of the scoped API.

11. **Add script logic** to extract the value of the u_password field from the response body.
 12. **Print the value** of the u_password field to the application log using the gs.info() method.

The script logic will look like this:

```
var response = request.execute();
gs.info(response.getBody());

var obj = JSON.parse(response.getBody());
var myPass = obj.result[0].u_password;
gs.info('u_password: ' + myPass);
```

[⊕ Show script](#)

13. Your code should look like this when complete. Select **Save**.

```
(function executeRule(current, previous /*null when async*/ ) {  
  
    if (current.item_type == "cmdb_ci_computer") {  
  
        var request = new sn_ws.RESTMessageV2();  
        request.setEndpoint('https://now.com/api/now/table/u\_password\_vault?sysparm\_query=u\_asset%3D'  
        + current.cmdb_ci.asset_tag + '&sysparm_fields=u_password&sysparm_limit=1');  
        request.setHttpMethod('GET');  
  
        //Eg. UserName="admin", Password="admin" for this code sample.  
        var user = 'admin';  
        var password = 'CHANGE THIS TO YOUR INSTANCE PASSWORD';  
  
        request.setBasicAuth(user, password);  
        request.setRequestHeader("Accept", "application/json");  
  
        var response = request.execute();  
        gs.info(response.getBody());  
  
        var obj = JSON.parse(response.getBody());  
        var myPass = obj.result[0].u_password;  
        gs.info('u_password: ' + myPass);  
    }  
})(current, previous);
```

It's time to test and debug the Business Rule.

Reminder: When testing, make sure to use one of the laptops that is part of the Loaner program. (i.e., Have passwords in the Password Vault table.)

Computers		
All > Asset tag in P1000503, P1000640, P1000815, P1000814, P1000517, P1000666, P1000871		
Name	Manufacturer	Operating System
Search	Search	Search
LNR - MacBook Air 13"	Apple	Mac OS 10 (OS/X)
LNR - MacBook Air 13"	Apple	Mac OS 10 (OS/X)
LNR - MacBook Pro 15"	Apple	Mac OS 10 (OS/X)

14. In the main ServiceNow window, open **Loaner Request > Open**.

15. **Open a Loaner Request record** and configure it:

State: **Ready for Pickup**

Item type: **Laptop**

Configuration item: **Set this to one of the laptops that is part of the Loaner program. (Hint: Type LNR)**

The screenshot shows the 'Loaner Request' form for record LNR002016. The following fields are highlighted with green boxes:

- State:** Ready for Pickup
- Configuration item:** LNR - MacBook Air 13"
- Item type:** Laptop

Other visible fields include:

- Number:** LNR002016
- * Requested for:** Abel Tuter
- * Start date:** 20 -01-28 20:14:22
- * End date:** 20 -02-07 20:14:24
- Assigned to:** (empty)
- Location to be used:** (empty)
- Depot:** Santa Clara

16. Select **Update**.

17. Check the Application Log by navigating to **System Logs > System Log > Application Logs**.

18. You should see the **password in the Message field**:

Created ▾	Level	Message
Search	Search	Search
20 -04-02 19:46:55	Information	u_password: mercury
20 -04-02 19:46:55	Information	{"result":[{"u_password":"mercury"}]}

19. Once the Business Rule is working, make the Business Rule inactive by **unchecking the Active** checkbox in ServiceNow Studio. Select **Save**.

Business Rule

Retrieved Credentials

Name

Retrieved Credentials

Table

Loaner Request [x_cdltd_loan... ▼]

Priority

100

Application

Loaner Request

Active

☐

Advanced

☒

20. **Copy the Business Rule script** to the Loaner Item Pick Up Email Notification. When copying, only the body of the script is needed.

```
(function executeRule(current, previous /*null when async*/ ) {
    if (current.item_type == "cmdb_ci_computer") {

        var request = new sn_ws.RESTMessageV2();
        request.setEndpoint('https://[redacted]
service-now.com/api/now/table/u_password_vault?sysparm_query=u_asset%3D'
+ current.cmdb_ci.asset_tag + '&sysparm_fields=u_password&
sysparm_limit=1');
        request.setHttpMethod('GET');

        //Eg. UserName="admin", Password="admin" for this code sample.
        var user = 'admin';
        var password = 'CHANGE THIS TO YOUR INSTANCE PASSWORD';

        request.setBasicAuth(user, password);
        request.setRequestHeader("Accept", "application/json");

        var response = request.execute();
        gs.info(response.getBody());

        var obj = JSON.parse(response.getBody());
        var myPass = obj.result[0].u_password;
        gs.info('u_password: ' + myPass);
    }
})(current, previous);
```

21. In ServiceNow Studio, navigate to the **Loaner Item Pickup Email Notification**. On the **What it will contain** tab, at the bottom of the Message HTML, add `<mail_script>` `</mail_script>` tags. **Paste the script** within the tags.
22. Remove the raw JSON response and keep only the parse JSON response. **Delete** `gs.info(response.getBody());`
23. Change the logging message to use the template.print method instead of a GlideSystem method. **Replace** `gs.info` **with** `template.print`.

Here is what the final result will look like:

```
Dear ${requested_for},

The ${item_type} (${cmdb_ci}) is available for pick up at ${depot}.

Your reservation is scheduled to begin ${start_date}. Please return the
item to the same location you picked it up by ${end_date}.

You will receive a return reminder notification.

<mail_script>

    if (current.item_type == "cmdb_ci_computer") {

        //Add your code here
        var request = new sn_ws.RESTMessageV2();

        request.setEndpoint('https://[redacted].service-now.com/
api/now/table/u_password_vault?sysparm_query=u_asset%3D'+
current.cmdb_ci.asset_tag +
'&sysparm_fields=u_password&sysparm_limit=1');
        request.setHttpMethod('GET');

        //Eg. UserName="admin", Password="admin" for this code sample.
        var user = 'admin';
        var password = '[redacted]';

        request.setBasicAuth(user, password);
        request.setRequestHeader("Accept", "application/json");

        var response = request.execute();

        var obj = JSON.parse(response.getBody());
        var myPass = obj.result[0].u_password;
        template.print('u_password: ' + myPass);

    }
</mail_script>
```

24. **Save** the Notification.

25. When prompted, allow ServiceNow to move the script into the Notification Email Scripts. Select **Yes**.

26. Test and debug the Email Notification.

Try to test without any hints!

If you need some assistance, select **+ Show testing hints** below.

[+ Show testing hints](#)

E. Complete the update set

1. In the main ServiceNow window, switch back to the **Loaner Request scope**.

2. In ServiceNow Studio, close all remaining tabs. Then **change the state of the module 7 update set to complete**.

(We've done this several times in class, so try to remember the steps on your own. If you need help, select Show steps below.)

[+ Show steps](#)

Outstanding! In this lab, you accomplished the following:

- You configured your Loaner Request application to be a web service consumer.
- You retrieved passwords from the web service provider based on asset tag value and sent the passwords to Loaner Item requesters via email.
- You completed the update set for module 7.

Congratulations! You have completed this lab.



ServiceNow recommendations



Good practices

- Always **clean up data before importing it** into ServiceNow.
- **Understand the format/structure/contents** of data *before* importing it into ServiceNow.
- Study and **understand what data a web service expects** to receive and the format and structure of the response.
- **Test REST message Functions** in ServiceNow *before* scripting actions on the function.
- **Do NOT practice** with the REST API Explorer **on a production instance**.



Module recap

Core concepts

- ServiceNow provides a number of **integration interfaces** to the platform.
- There are **many factors to consider** when selecting an integration method.
- **Import data** into ServiceNow using one of these integration methods to avoid re-creating records.

Real world use cases

- **Why** would you use these capabilities?
- **When** would you use these capabilities?
- **How often** would you use these capabilities?

Discuss: Why, when, and how often would you use the capabilities shown in this module?

